

DATABASE GROUP ASSIGNMENT

HOTEL MANAGEMENT SYSTEM

Normalization Process Documentation

Using MySQL and MongoDB

No	Full Name	Student ID
1	Beka Solomon	ETS 0242/17
2	Deborah Mesfin	ETS 0434/17
3	Biruk Molla	ETS 0336/17
4	Abreham Teshale	ETS 0076/17
5	Awach Garang	ETS 1869/17
6	Gakeer Deng	ETS 1865/17

Group	Group 7
Submitted To	Mr. Yaynshet Medhin
Date	May 2026

Table of Contents

§	Section	Page
1	Introduction & Objectives	3
2	Unnormalized Form (UNF)	3
3	First Normal Form (1NF)	4
4	Second Normal Form (2NF)	5
5	Third Normal Form (3NF)	6
6	Boyce-Codd Normal Form (BCNF)	7
7	Food & Service Ordering Normalization	8
8	Final Schema with Sample Data	9
9	ER Diagram (Textual Representation)	11
10	Summary & Conclusion	12

1. Introduction & Objectives

This document presents the complete database normalization process for the **Addis Grand Hotel Management System**. The goal of normalization is to organize data efficiently, eliminate redundancy, and prevent anomalies. We apply normalization rules progressively from Unnormalized Form (UNF) through to Boyce-Codd Normal Form (BCNF), demonstrating each transformation with functional dependencies, before/after tables, and concrete examples.

Normalization Objectives

- Eliminate data redundancy and duplication
- Remove insert, update, and delete anomalies
- Ensure data integrity and consistency
- Produce a clean, scalable relational schema
- Support both MySQL (relational) and MongoDB (NoSQL) implementations

2. Unnormalized Form (UNF)

Starting point — a single flat table with all hotel data

All hotel data is stored in one giant table. This causes severe problems:

Bookin gID	GuestName	Phone	Email	RoomNumb ers	CheckIn	CheckOu t	TotalA mt	RoomTyp es	Prices	OrderedItems
B001	Biruk Molla	0912345678	biruk@gmail.com	A-101, B-205	2026-04-20	2026-04-25	8500	Double, Deluxe	1200, 1500	Chicken, Fries, Soft Drink
B002	Deborah Mesfin	0923456789	debo@yahoo.com	C-301	2026-05-01	2026-05-03	3000	Suite	1500	Coffee, Cake
B003	Abreham Teshale	0934567890	abri@email.com	A-102, A-103	2026-05-10	2026-05-15	12000	Double, Double	1200, 1200	Breakfast, Juice

Problems (Before)

- ✗ Multi-valued attributes (multiple rooms, items in one cell)
- ✗ Repeating groups — RoomNumbers and OrderedItems repeat
- ✗ Massive data redundancy — guest info duplicated per booking
- ✗ Update anomaly — changing a phone number requires many updates
- ✗ Insert anomaly — cannot add a room without a booking
- ✗ Delete anomaly — deleting a booking loses guest data

Solution (After)

- ✓ Apply 1NF — make every attribute atomic
- ✓ One value per cell, no repeating groups
- ✓ Separate rows for each room in a booking
- ✓ Prepare for further normalization in 2NF and 3NF

3. First Normal Form (1NF)

Rule: Every attribute must be atomic; no repeating groups

Problems (Before)

- ✗ RoomNumbers contains "A-101, B-205" — not atomic
- ✗ OrderedItems contains "Chicken, Fries, Soft Drink" — not atomic
- ✗ RoomTypes contains "Double, Deluxe" — not atomic
- ✗ No clear primary key for the table

Solution (After)

- ✓ Split multi-valued cells into separate rows
- ✓ Each row represents one (BookingID, RoomNumber) combination
- ✓ Composite primary key: (BookingID, RoomNumber)
- ✓ All attribute values are now single-valued (atomic)

1NF Table (after splitting multi-valued attributes):

Booking ID	GuestName	Phone	Email	RoomNo	CheckIn	CheckOut	TotalAmt	RoomType	PricePerNight
B001	Biruk Molla	0912345678	biruk@gmail.com	A-101	2026-04-20	2026-04-25	8500	Double	1200
B001	Biruk Molla	0912345678	biruk@gmail.com	B-205	2026-04-20	2026-04-25	8500	Deluxe	1500
B002	Deborah Mesfin	0923456789	debo@yahoo.com	C-301	2026-05-01	2026-05-03	3000	Suite	1500
B003	Abreham Teshale	0934567890	abri@email.com	A-102	2026-05-10	2026-05-15	12000	Double	1200
B003	Abreham Teshale	0934567890	abri@email.com	A-103	2026-05-10	2026-05-15	12000	Double	1200

Functional Dependencies

(BookingID, RoomNo) → GuestName, Phone, Email, CheckIn, CheckOut, TotalAmt [Composite PK]

BookingID → GuestName, Phone, Email, CheckIn, CheckOut, TotalAmt [Partial dependency – problem!]

RoomNo → RoomType, PricePerNight [Partial dependency – problem!]

■ 1NF still has partial dependencies — RoomType and Price depend only on RoomNo, not on the full composite key. This violates 2NF.

4. Second Normal Form (2NF)

Rule: Must be in 1NF and have no partial dependencies on the primary key

Problems (Before)

- ✗ GuestName, Phone, Email depend only on BookingID — partial dependency
- ✗ RoomType, PricePerNight depend only on RoomNo — partial dependency
- ✗ Guest data is repeated for every room in the same booking

Solution (After)

- ✓ Split into two tables: Booking and Room
- ✓ Booking table: BookingID (PK), GuestName, Phone, Email, CheckIn, CheckOut, TotalAmt
- ✓ Room table: RoomNo (PK), RoomType, PricePerNight
- ✓ All non-key attributes now fully depend on the entire primary key

BEFORE (1NF) — Partial Dependencies Present:

BookingID	GuestName	Phone	Email	RoomNo	CheckIn	CheckOut	RoomType	Price
B001	Biruk Molla	0912...	biruk@gmail.com	A-101	2026-04-20	2026-04-25	Double	1200
B001	Biruk Molla	0912...	biruk@gmail.com	B-205	2026-04-20	2026-04-25	Deluxe	1500
B002	Deborah Mesfin	0923...	debo@yahoo.com	C-301	2026-05-01	2026-05-03	Suite	1500

AFTER — Two Separate 2NF Tables:

Booking Table (BookingID is PK):

BookingID	GuestName	Phone	Email	CheckIn	CheckOut	TotalAmt
B001	Biruk Molla	0912345678	biruk@gmail.com	2026-04-20	2026-04-25	8500
B002	Deborah Mesfin	0923456789	debo@yahoo.com	2026-05-01	2026-05-03	3000
B003	Abreham Teshale	0934567890	abri@email.com	2026-05-10	2026-05-15	12000

Room Table (RoomNo is PK):

RoomNo	RoomType	PricePerNight
A-101	Double	1200
B-205	Deluxe	1500
C-301	Suite	1800
A-102	Double	1200
A-103	Double	1200

Functional Dependencies

- BookingID → GuestName, Phone, Email, CheckIn, CheckOut, TotalAmt [Full dependency ✓]
- RoomNo → RoomType, PricePerNight [Full dependency ✓]
- Still a problem: GuestName → Phone, Email (transitive dependency) [Violates 3NF!]

5. Third Normal Form (3NF)

Rule: Must be in 2NF and have no transitive dependencies

Problems (Before)

- ✗ In the Booking table: GuestName → Phone, Email (transitive)
- ✗ Phone and Email depend on the Guest, not on the BookingID
- ✗ If a guest changes their email, every booking row must be updated
- ✗ Cannot store guest info without a booking — insert anomaly

Solution (After)

- ✓ Create a separate Guest table with its own GuestID (PK)
- ✓ Booking table references Guest via GuestID (FK)
- ✓ Also separate RoomType details from Room table
- ✓ All non-key attributes now depend directly on their PK only

Final 3NF Tables:

Guest Table (guest_id PK):

guest_id	full_name	phone	email	address	nationality
G001	Biruk Molla	0912345678	biruk@gmail.com	Addis Ababa	Ethiopian
G002	Deborah Mesfin	0923456789	debo@yahoo.com	Addis Ababa	Ethiopian
G003	Abreham Teshale	0934567890	abri@email.com	Bahir Dar	Ethiopian
G004	Awach Garang	0945678901	awach@email.com	Juba	South Sudanese

Booking Table (booking_id PK, guest_id FK):

booking_id	guest_id	check_in	check_out	total_amount	status
B001	G001	2026-04-20	2026-04-25	8500	Completed
B002	G002	2026-05-01	2026-05-03	3000	Completed
B003	G003	2026-05-10	2026-05-15	12000	Active
B004	G004	2026-06-01	2026-06-05	7200	Pending

RoomType Table (type_id PK):

type_id	type_name	price_per_night	max_occupancy
T001	Single	800	1
T002	Double	1200	2
T003	Deluxe	1500	2
T004	Suite	1800	4

Room Table (room_id PK, type_id FK):

room_id	room_number	type_id	status	floor_number
R001	A-101	T002	Available	1
R002	B-205	T003	Occupied	2

room_id	room_number	type_id	status	floor_number
R003	C-301	T004	Available	3
R004	A-102	T002	Available	1

Functional Dependencies

guest_id → full_name, phone, email, address, nationality [Direct, no transitive ✓]

booking_id → guest_id, check_in, check_out, total_amount, status [Direct ✓]

type_id → type_name, price_per_night, max_occupancy [Direct ✓]

room_id → room_number, type_id, status, floor_number [Direct ✓]

6. Boyce-Codd Normal Form (BCNF)

Rule: For every non-trivial FD $X \rightarrow Y$, X must be a superkey

BCNF is a stricter version of 3NF. A table is in BCNF if, for every functional dependency $X \rightarrow Y$, X is a superkey (i.e., X can uniquely identify every row). We verify each table:

Table	Functional Dependencies	Determinant is Superkey?	BCNF?
Guest	guest_id $\rightarrow$ all attributes	guest_id is PK ✓	Yes ✓
Booking	booking_id $\rightarrow$ all attributes	booking_id is PK ✓	Yes ✓
RoomType	type_id $\rightarrow$ all attributes	type_id is PK ✓	Yes ✓
Room	room_id $\rightarrow$ all attributes	room_id is PK ✓	Yes ✓
BookingRoom	(booking_id, room_id) $\rightarrow$ —	Composite PK ✓	Yes ✓
Payment	payment_id $\rightarrow$ all attributes	payment_id is PK ✓	Yes ✓
FoodOrder	order_id $\rightarrow$ all attributes	order_id is PK ✓	Yes ✓
OrderItem	(order_id, item_id) $\rightarrow$ qty	Composite PK ✓	Yes ✓

All determinants in every table are candidate keys. Therefore the schema is fully in BCNF. No further decomposition is needed.

7. Food & Service Ordering Normalization

Normalizing the OrderedItems attribute from UNF

In the original UNF table, ordered items like "Chicken, Fries, Soft Drink" were stored in a single cell. This section shows how food and service orders are properly normalized.

UNF Problem — Items in one cell:

BookingID	OrderedItems	Total
B001	Chicken, Fries, Soft Drink	450
B002	Coffee, Cake	180

Solution — Three normalized tables:

MenuItem Table (item_id PK):

item_id	item_name	category	price
M001	Chicken	Main Course	180
M002	Fries	Side Dish	60
M003	Soft Drink	Beverage	40
M004	Coffee	Beverage	50
M005	Cake	Dessert	90
M006	Breakfast Buffet	Main Course	200
M007	Juice	Beverage	60

FoodOrder Table (order_id PK, booking_id FK):

order_id	booking_id	order_date	order_time	total_amount	status
FO001	B001	2026-04-21	12:30	450	Delivered
FO002	B001	2026-04-22	19:00	280	Delivered
FO003	B002	2026-05-01	09:00	180	Delivered

OrderItem Table — Junction (order_id + item_id composite PK):

order_id	item_id	quantity	unit_price	subtotal
FO001	M001	1	180	180
FO001	M002	2	60	120
FO001	M003	2	40	80
FO002	M004	1	50	50
FO002	M005	1	90	90
FO003	M006	1	200	200

Functional Dependencies

```
item_id → item_name, category, price [MenuItem – BCNF ✓]
order_id → booking_id, order_date, total_amount [FoodOrder – BCNF ✓]
(order_id, item_id) → quantity, unit_price, subtotal [OrderItem – BCNF ✓]
```

8. Final Schema with Sample Data

Complete BCNF schema — 8 tables

BookingRoom Junction Table (booking_id + room_id composite PK):

booking_id	room_id
B001	R001
B001	R002
B002	R003
B003	R001
B003	R004

Payment Table (payment_id PK, booking_id FK):

payment_id	booking_id	amount	payment_date	payment_method	status
P001	B001	4250	2026-04-20	Credit Card	Paid
P002	B001	4250	2026-04-25	Cash	Paid
P003	B002	3000	2026-05-01	Mobile Money	Paid
P004	B003	6000	2026-05-10	Bank Transfer	Partial

Complete Final Schema Summary:

#	Table Name	Primary Key	Attributes
1	RoomType	type_id (PK)	type_name, price_per_night, max_occupancy, description
2	Room	room_id (PK)	room_number, type_id (FK), status, floor_number
3	Guest	guest_id (PK)	full_name, phone, email, address, nationality, id_number
4	Booking	booking_id (PK)	guest_id (FK), check_in, check_out, total_amount, status
5	BookingRoom	(booking_id, room_id) PK	Junction table — many-to-many
6	Payment	payment_id (PK)	booking_id (FK), amount, date, method, status
7	MenuItem	item_id (PK)	item_name, category, price
8	FoodOrder	order_id (PK)	booking_id (FK), order_date, total_amount, status
9	OrderItem	(order_id, item_id) PK	quantity, unit_price, subtotal

9. Entity-Relationship Overview

Relationships between all entities in the final BCNF schema

Entity Relationships:

Entity A	Entity B	Relationship	Description	Implementation
Guest	Booking	One-to-Many	One guest can make many bookings	guest_id (FK in Booking)
Booking	Room	Many-to-Many	One booking can include many rooms; a room can be in many bookings	BookingRoom junction
RoomType	Room	One-to-Many	One room type applies to many rooms	type_id (FK in Room)
Booking	Payment	One-to-Many	One booking can have multiple payments	booking_id (FK in Payment)
Booking	FoodOrder	One-to-Many	One booking can have many food orders	booking_id (FK in FoodOrder)
FoodOrder	MenuItem	Many-to-Many	One order can have many items; an item can be in many orders	OrderItem junction

Schema Diagram (Textual):

GUEST (guest_id PK) ■■■■■■■■■■■■■■■■■■■■ BOOKING (booking_id PK, guest_id FK) ■ ■ ■ ■
BOOKINGROOM PAYMENT (booking_id, (payment_id PK, room_id) booking_id FK) ■ ROOM (room_id
PK, type_id FK) ■ ROOMTYPE (type_id PK) BOOKING ■■■■ FOODORDER (order_id PK, booking_id
FK) ■ ORDERITEM (order_id + item_id PK) ■ MENUITEM (item_id PK)

10. Summary & Conclusion

Step	Problem Solved	Result
UNF → 1NF	Removed multi-valued attributes & repeating groups	All cells atomic; composite PK added
1NF → 2NF	Removed partial dependencies	Split into Booking and Room tables
2NF → 3NF	Removed transitive dependencies	Created separate Guest and RoomType tables
3NF → BCNF	Verified all determinants are superkeys	All 9 tables confirmed in BCNF
Extra	Normalized Food & Service Ordering	Added MenuItem, FoodOrder, OrderItem tables

Benefits Achieved

- Zero data redundancy — each fact stored exactly once
- No update anomalies — changing guest email requires only one update
- No insert anomalies — rooms and guests can be added independently
- No delete anomalies — deleting a booking does not lose guest data
- Referential integrity enforced through foreign keys
- Schema supports both MySQL (relational) and MongoDB (document model)
- Scalable design — easy to add new entities (Staff, Housekeeping, etc.)

The Addis Grand Hotel Management System database has been successfully normalized from a single flat Unnormalized Form table to a clean, fully decomposed **Boyce-Codd Normal Form (BCNF)** schema consisting of 9 interrelated tables. This design eliminates all forms of data anomalies, ensures integrity, and provides a solid foundation for implementation using both MySQL and MongoDB.

Group 7 — Department of Software Engineering | Submitted to: Mr. Yaynshet Medhin | May 2026